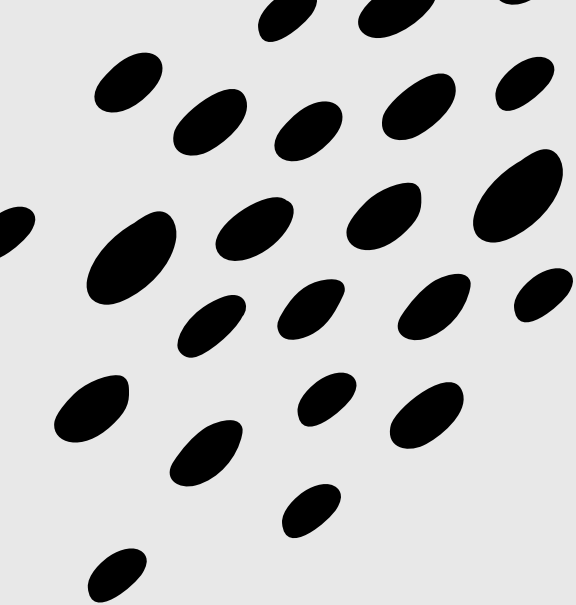
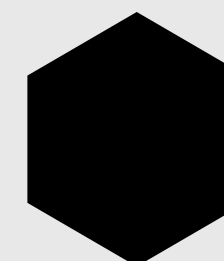


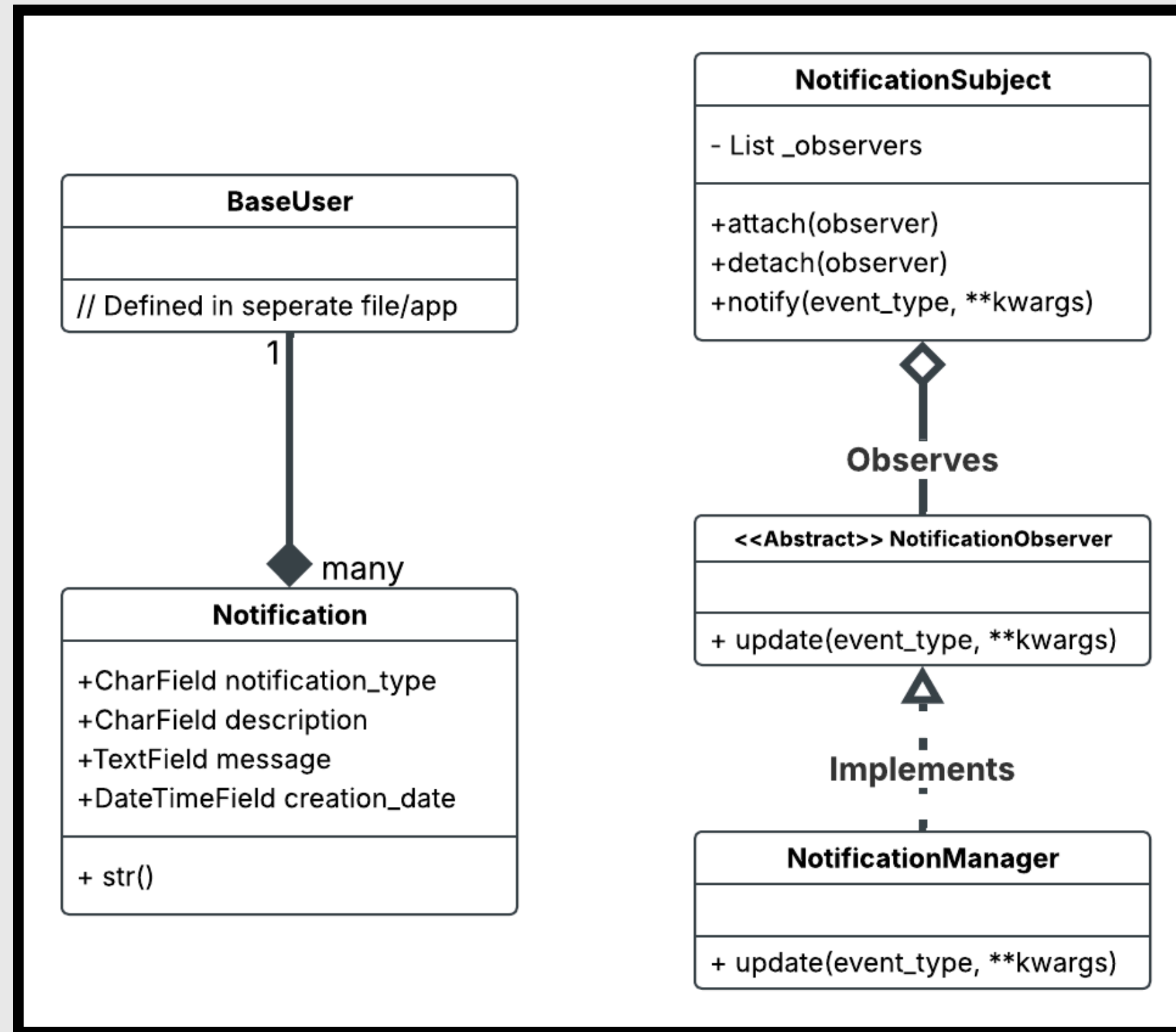
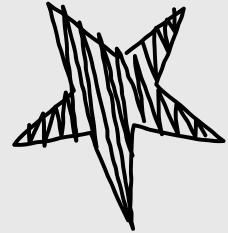
KosmoBook

OBSERVER

JAMARI R | SHANIA P
TYRONE B



CLASS DIAGRAM

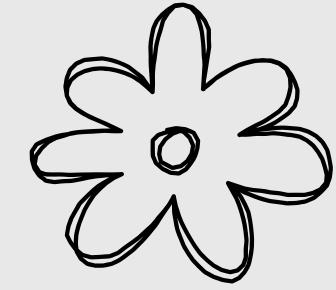


OBSERVER IMPLEMENTATION

```
'''This is created/called to create a notification type object when a
media/comment creation function is called such as _create_photo_upload_notification()'''
class Notification(models.Model):
    NOTIFICATION_TYPES = [
        ('video_upload', 'Video Uploaded'),
        ('photo_upload', 'Photo Uploaded'),
        ('video_liked', 'Video Liked'),
        ('photo_liked', 'Photo Liked'),
        ('comment_added', 'Comment Added'),
        ('appointment_booked', 'Appointment Booked'),
        ('appointment_reminder', 'Appointment Reminder'),
        ('follow', 'New Follower'),
        ('test', 'Test Notification')
    ]

    user = models.ForeignKey(BaseUser, on_delete=models.CASCADE, related_name='notifications')
    description = models.CharField(max_length=200)
    message = models.TextField()
    notification_type = models.CharField(max_length=50, choices=NOTIFICATION_TYPES)
    creation_date = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user.username}: {self.description}"
```



```
'''Responsible for notifying observers on its state data. When update(notify) is called,
sends the relevant info to NotificationManager'''
class NotificationSubject:
    def __init__(self):
        self._observers = []
        # adds observers/subscribers (users) to the notification system.
        # For example, manager = NotificationManager(), subject.attach(manager)
    def attach(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)
        # removes observers/subscribers (users) to the notification system
    def detach(self, observer):
        self._observers.remove(observer)
        # Notify calls on each observers update function and passes through the required parameters.
        # subject.notify('photo_uploaded', etc...)
        # **kwargs is flexible and helps pass more data that may be needed to create the notification
    def notify(self, event_type, **kwargs):
        for observer in self._observers:
            observer.update(event_type, **kwargs)
```

```
'''The interface for the observer/subscriber, ensures
everything passed has an update function'''
class NotificationObserver:
    def update(self, event_type, **kwargs):
```

'''The NotificationManager is what actually creates and sends out the notification to the user'''

```
class NotificationManager(NotificationObserver):
    def update(self, event_type, **kwargs):
        # This method is called when observed subjects change and
        # checks for what type of notification it should send out
        if event_type == 'video_uploaded':
            self._create_video_upload_notification(**kwargs)
        elif event_type == 'photo_uploaded':
        elif event_type == 'appointment_booked':
        elif event_type == 'video_liked':
        elif event_type == 'photo_liked':
        elif event_type == 'comment_added':
#ADD FUTURE EXISTING EVENTS LIKE: APPOINTMENT UPDATED

# Creates the actual notification (object) that is sent out to a user
    def _create_photo_upload_notification(self, photo, **kwargs):

    def _create_photo_like_notification(self, photo, liker, **kwargs):

    def _create_comment_notification(self, comment, **kwargs):
```

ADVANTAGES

- CAN EASILY ADD/REMOVE/MODIFY DIFFERENT TYPES OF NOTIFICATIONS DURING OR AFTER DEVELOPMENT
- CENTRALIZES LOGIC FOR EASY MANAGEMENT AND DIFFERENT USE CASES (ANOUNCMENTS, ALERTS, UPDATES, ETC)

THANKS!

